

Transportation and Shipping

Instructor: Sairam Sriraman

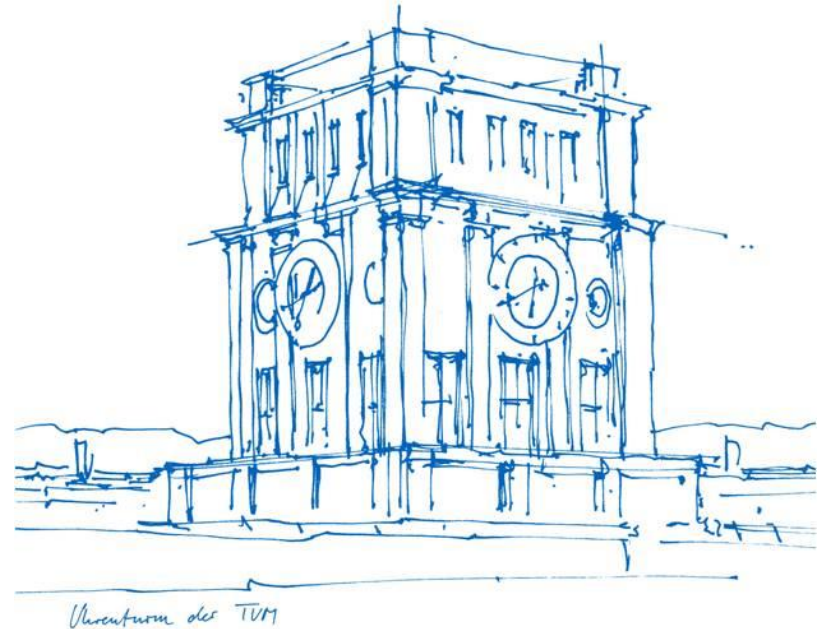
Examiner: Prof. Dr. David Wuttke

Technische Universität München

School of Management

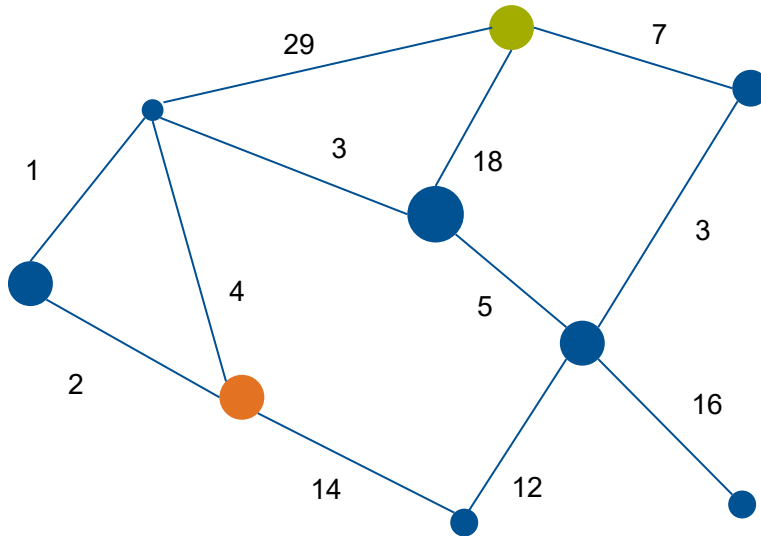
Campus Heilbronn

Heilbronn, 2026



Shortest Path Problem

Shortest Path Problem



Plant location



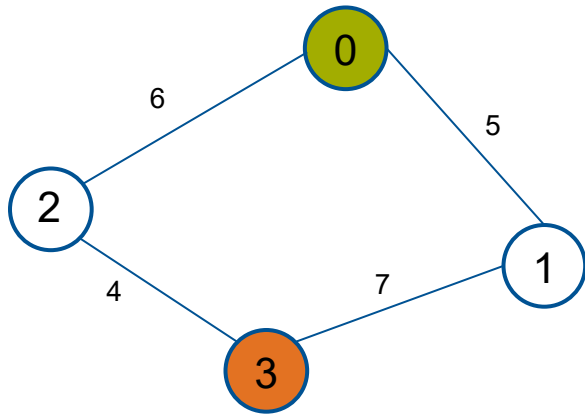
Customer location

- What is the shortest path from your plant to one customer?

Dijkstra's Algorithm

1. Mark all nodes unvisited. Create a set of all the unvisited nodes called the *unvisited set*.
2. Assign to every node a tentative distance value: set it to zero for our initial node and to infinity for all other nodes. Set the initial node as current.
3. For the current node, consider all of its unvisited neighbours and calculate their *tentative* distances through the current node. Compare the newly calculated *tentative* distance to the current assigned value and assign the smaller one.
4. When we are done considering all of the unvisited neighbours of the current node, mark the current node as visited and remove it from the *unvisited set*. A visited node will never be checked again.
5. If the destination node has been marked visited (when planning a route between two specific nodes) or if the smallest tentative distance among the nodes in the *unvisited set* is infinity (when planning a complete traversal; occurs when there is no connection between the initial node and remaining unvisited nodes), then stop. The algorithm has finished.
6. Otherwise, select the unvisited node that is marked with the smallest tentative distance, set it as the new "current node", and go back to step 3.

Dijkstra's Algorithm **EXAMPLE**

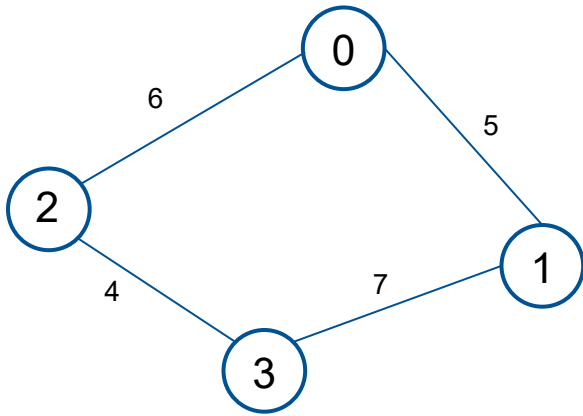


Plant location

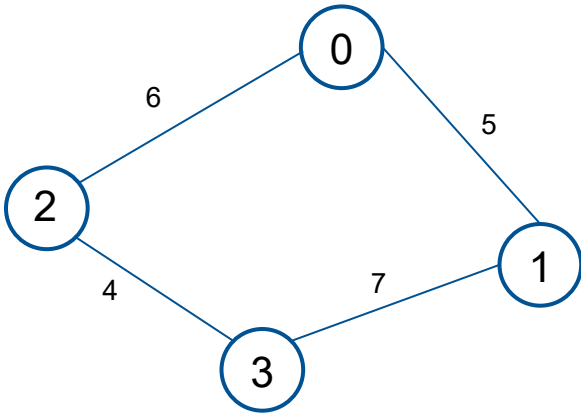


Customer location

Dijkstra's Algorithm **EXAMPLE**



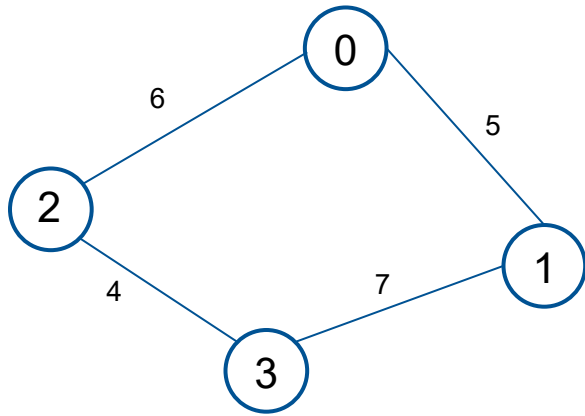
Dijkstra's Algorithm **EXAMPLE**



1. Mark all nodes unvisited. Create a set of all the unvisited nodes called the *unvisited set*.

$$U = \{0,1,2,3\}$$

Dijkstra's Algorithm **EXAMPLE**

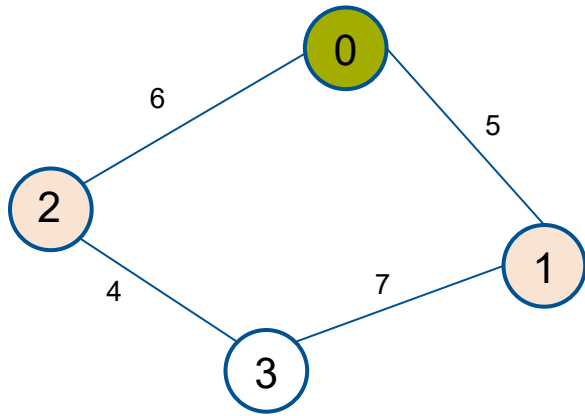


2. Assign to every node a tentative distance value: set it to zero for our initial node and to infinity for all other nodes. Set the initial node as current.

$$U = \{0,1,2,3\}$$

Node	Distance
0	0
1	∞
2	∞
3	∞

Dijkstra's Algorithm **EXAMPLE**

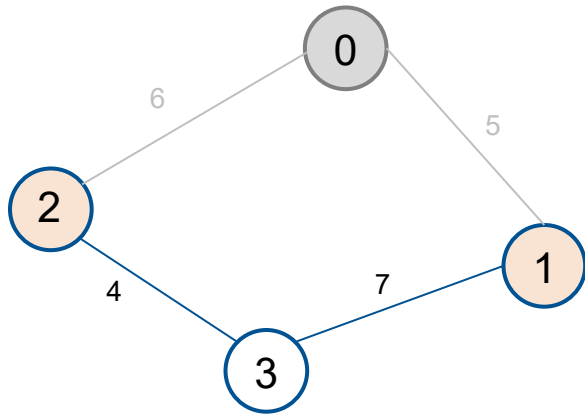


- For the current node, consider all of its unvisited neighbours and calculate their *tentative* distances through the current node. Compare the newly calculated *tentative* distance to the current assigned value and assign the smaller one.

$$U = \{0,1,2,3\}$$

Node	Distance
0	0
1	5
2	6
3	∞

Dijkstra's Algorithm **EXAMPLE**

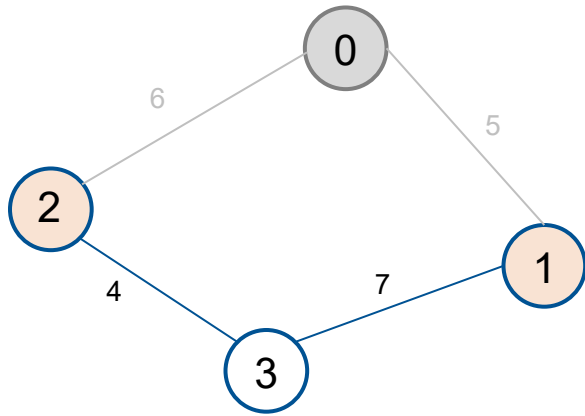


4. When we are done considering all of the unvisited neighbours of the current node, mark the current node as visited and remove it from the *unvisited set*. A visited node will never be checked again.

$$U = \{1,2,3\}$$

Node	Distance
0	0
1	5
2	6
3	∞

Dijkstra's Algorithm **EXAMPLE**

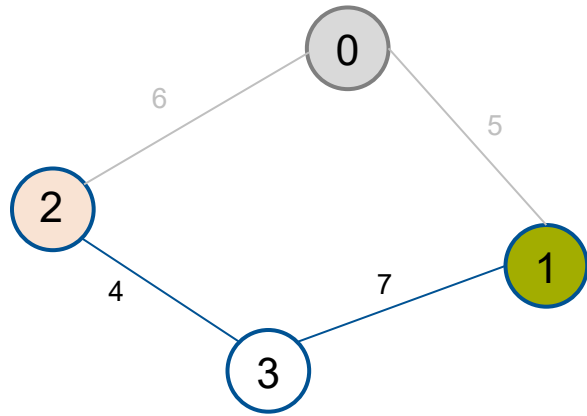


5. If the destination node has been marked visited (when planning a route between two specific nodes) or if the smallest tentative distance among the nodes in the unvisited set is infinity, then stop. The algorithm has finished.

$$U = \{1,2,3\}$$

Node	Distance
0	0
1	5
2	6
3	∞

Dijkstra's Algorithm **EXAMPLE**

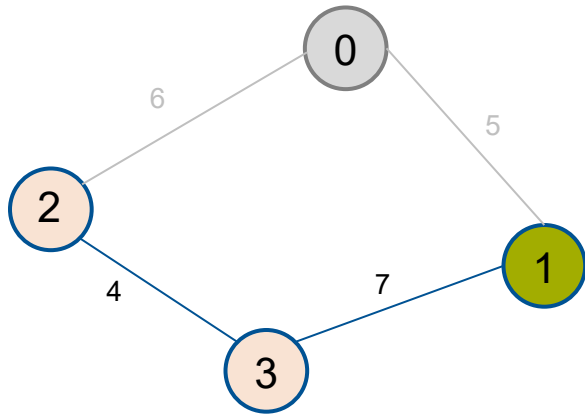


6. Otherwise, select the unvisited node that is marked with the smallest tentative distance, set it as the new "current node", and go back to step 3.

$$U = \{1,2,3\}$$

Node	Distance
0	0
1	5
2	6
3	∞

Dijkstra's Algorithm **EXAMPLE**

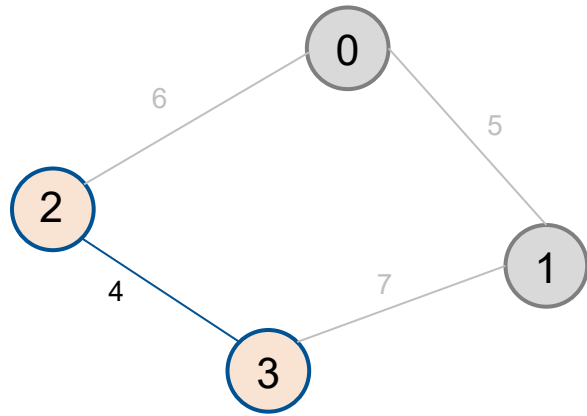


3. For the current node, consider all of its unvisited neighbours and calculate their *tentative* distances through the current node. Compare the newly calculated *tentative* distance to the current assigned value and assign the smaller one.

$$U = \{1,2,3\}$$

Node	Distance
0	0
1	5
2	6
3	12

Dijkstra's Algorithm **EXAMPLE**

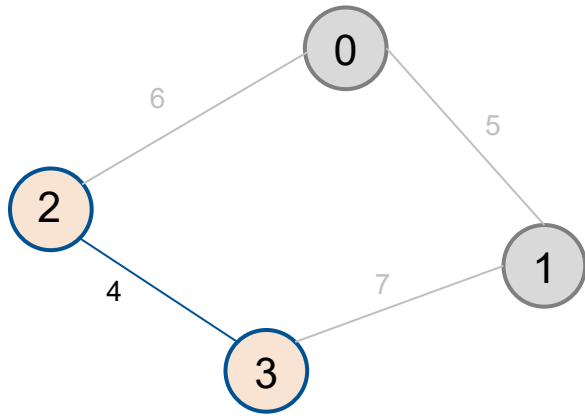


4. When we are done considering all of the unvisited neighbours of the current node, mark the current node as visited and remove it from the *unvisited set*. A visited node will never be checked again.

$$U = \{2,3\}$$

Node	Distance
0	0
1	5
2	6
3	12

Dijkstra's Algorithm **EXAMPLE**

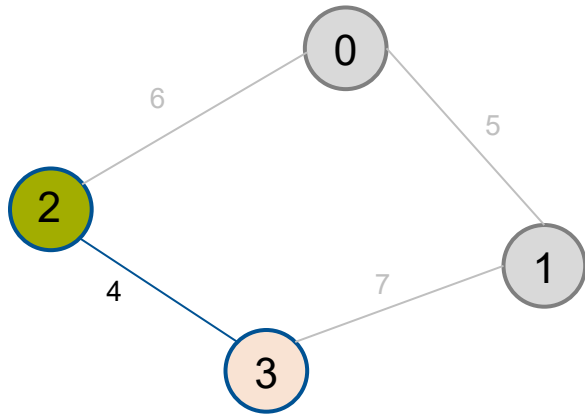


5. If the destination node has been marked visited (when planning a route between two specific nodes) or if the smallest tentative distance among the nodes in the unvisited set is infinity, then stop. The algorithm has finished.

$$U = \{2,3\}$$

Node	Distance
0	0
1	5
2	6
3	12

Dijkstra's Algorithm **EXAMPLE**

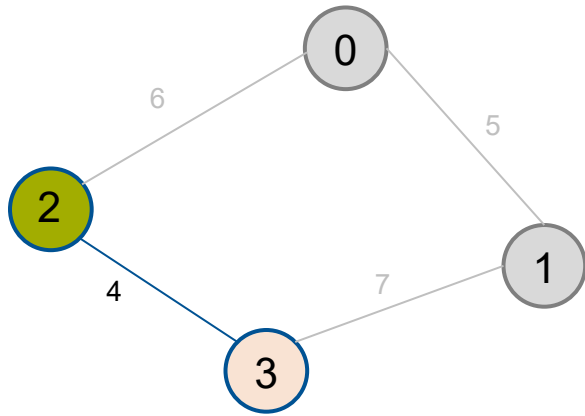


6. Otherwise, select the unvisited node that is marked with the smallest tentative distance, set it as the new "current node", and go back to step 3.

$$U = \{2,3\}$$

Node	Distance
0	0
1	5
2	6
3	12

Dijkstra's Algorithm **EXAMPLE**

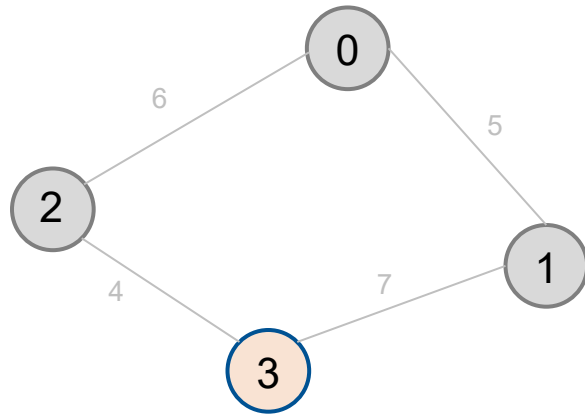


3. For the current node, consider all of its unvisited neighbours and calculate their *tentative* distances through the current node. Compare the newly calculated *tentative* distance to the current assigned value and assign the smaller one.

$$U = \{2,3\}$$

Node	Distance
0	0
1	5
2	6
3	10

Dijkstra's Algorithm **EXAMPLE**

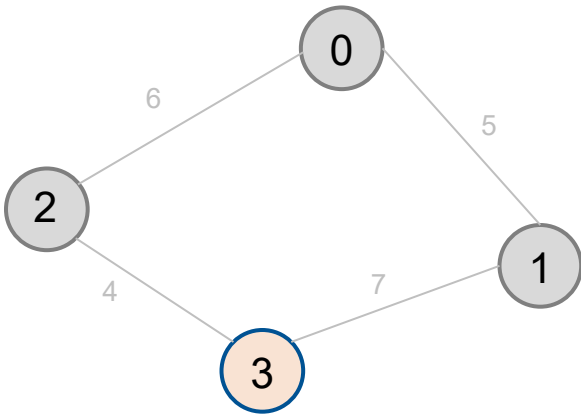


4. When we are done considering all of the unvisited neighbours of the current node, mark the current node as visited and remove it from the *unvisited set*. A visited node will never be checked again.

$$U = \{3\}$$

Node	Distance
0	0
1	5
2	6
3	10

Dijkstra's Algorithm **EXAMPLE**

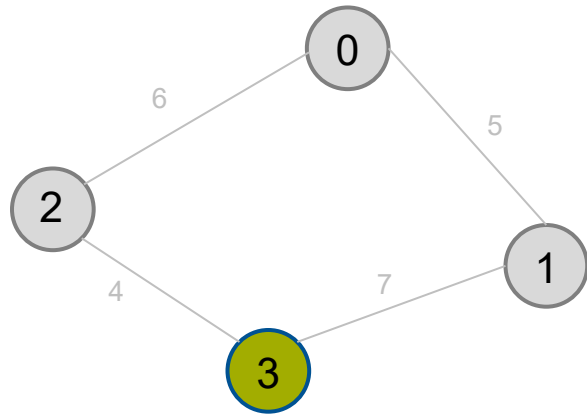


5. If the destination node has been marked visited (when planning a route between two specific nodes) or if the smallest tentative distance among the nodes in the unvisited set is infinity, then stop. The algorithm has finished.

$$U = \{3\}$$

Node	Distance
0	0
1	5
2	6
3	10

Dijkstra's Algorithm **EXAMPLE**

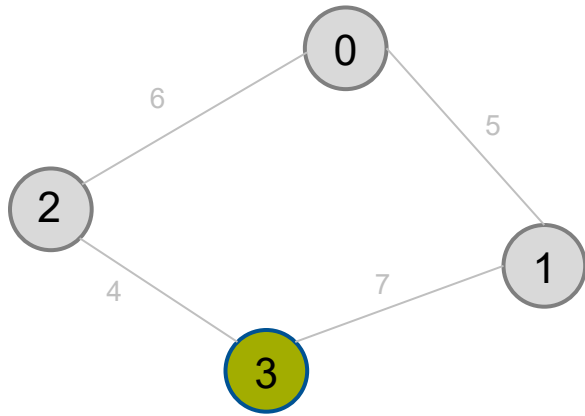


- Otherwise, select the unvisited node that is marked with the smallest tentative distance, set it as the new "current node", and go back to step 3.

$$U = \{3\}$$

Node	Distance
0	0
1	5
2	6
3	10

Dijkstra's Algorithm **EXAMPLE**

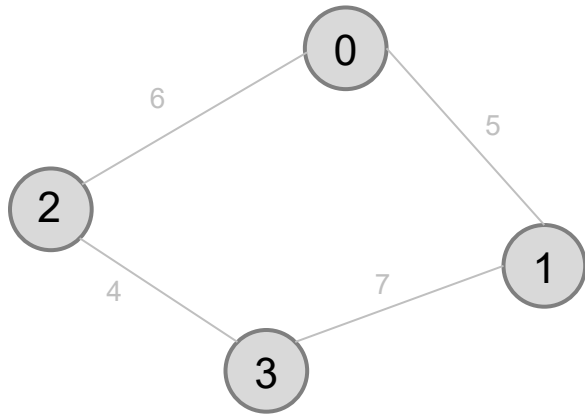


- For the current node, consider all of its unvisited neighbours and calculate their *tentative* distances through the current node. Compare the newly calculated *tentative* distance to the current assigned value and assign the smaller one.

$$U = \{3\}$$

Node	Distance
0	0
1	5
2	6
3	10

Dijkstra's Algorithm **EXAMPLE**

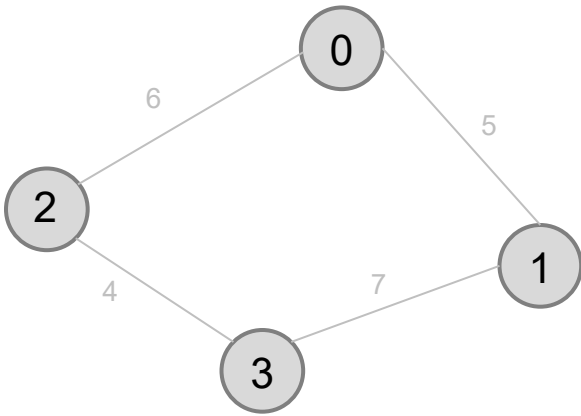


4. When we are done considering all of the unvisited neighbours of the current node, mark the current node as visited and remove it from the *unvisited set*. A visited node will never be checked again.

$$U = \{\}$$

Node	Distance
0	0
1	5
2	6
3	10

Dijkstra's Algorithm **EXAMPLE**

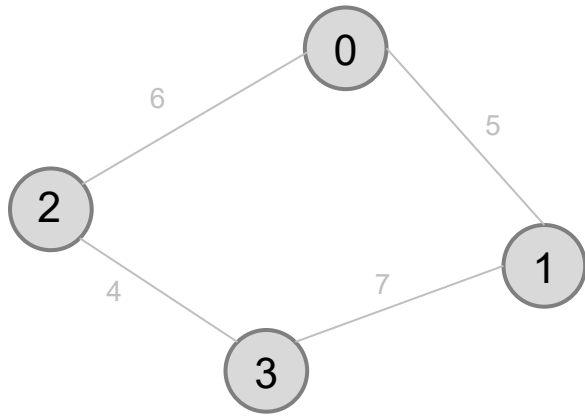


5. If the destination node has been marked visited (when planning a route between two specific nodes) or if the smallest tentative distance among the nodes in the unvisited set is infinity, then stop. The algorithm has finished.

$$U = \{\}$$

Node	Distance
0	0
1	5
2	6
3	10

Dijkstra's Algorithm **EXAMPLE**



5. If the destination node has been marked visited (when planning a route between two specific nodes) or if the smallest tentative distance among the nodes in the unvisited set is infinity, then stop. The algorithm has finished.

$$U = \{\}$$

Node	Distance
0	0
1	5
2	6
3	10

Solution: the shortest path from 0 to 3 is via 2. The total length is 10.

Dijkstra's Algorithm Running Time

The running time is limited by $\mathcal{O}(|V|^2)$.

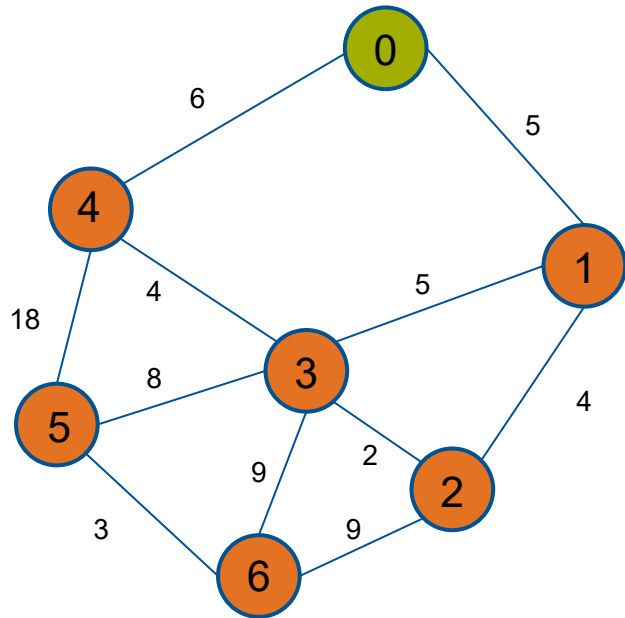
Depending on the data structure, faster running times are possible.

Optimal running time of $\mathcal{O}(|V| \log(|V|) + |E|)$ can be reached with Fibonacci-Heaps.

How are those implemented? Outside the scope of this course :)

Travelling Salesmen Problem (TSP)

(Symmetric) Traveling Salesmen Problem (TSP)



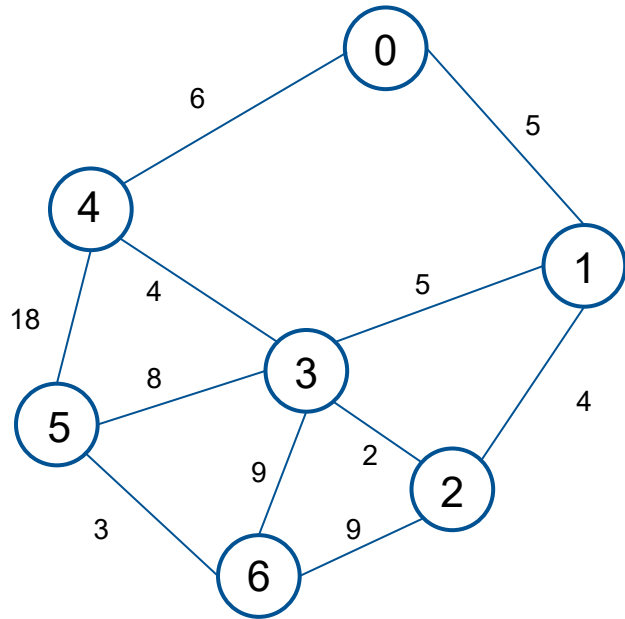
Production location



Customer location

TSP: Find the shortest path to visit each customer exactly once.

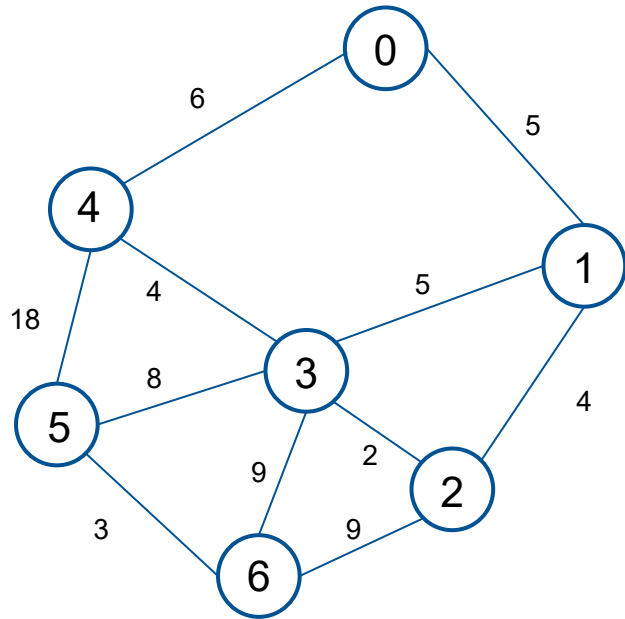
(Symmetric) Traveling Salesmen Problem (TSP)



V = Set of vertices (=all location nodes including plant location)

E = Set of all edges (=all connections between a pair of nodes)

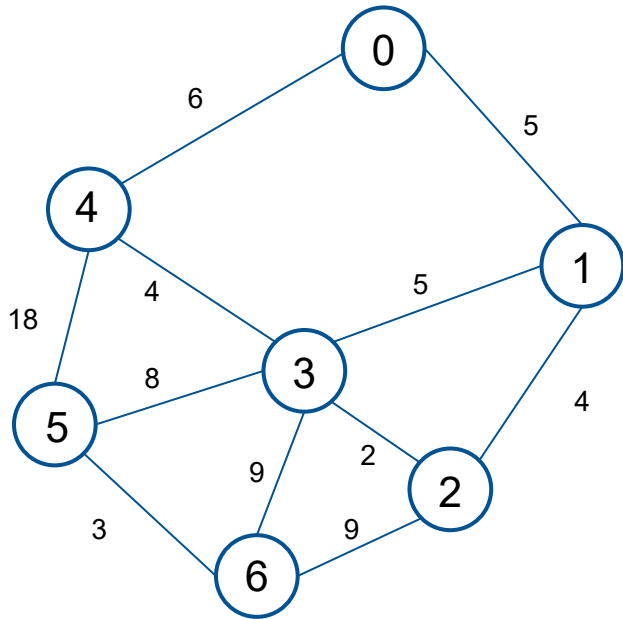
(Symmetric) Traveling Salesmen Problem (TSP)



$$V = \{0,1,2,3,4,5,6\}$$

$$E = \{ (0,1), (1,0), (0,4), (4,0), \\ (1,2), (2,1), (1,3), (3,1), \\ (2,3), (3,2), (2,6), (6,2), \\ (3,4), (4,3), (3,5), (5,3), (3,6), (6,3), \\ (4,5), (5,4), \\ (5,6), (6,5) \}$$

(Symmetric) Traveling Salesmen Problem (TSP)



$$V = \{0,1,2,3,4,5,6\}$$

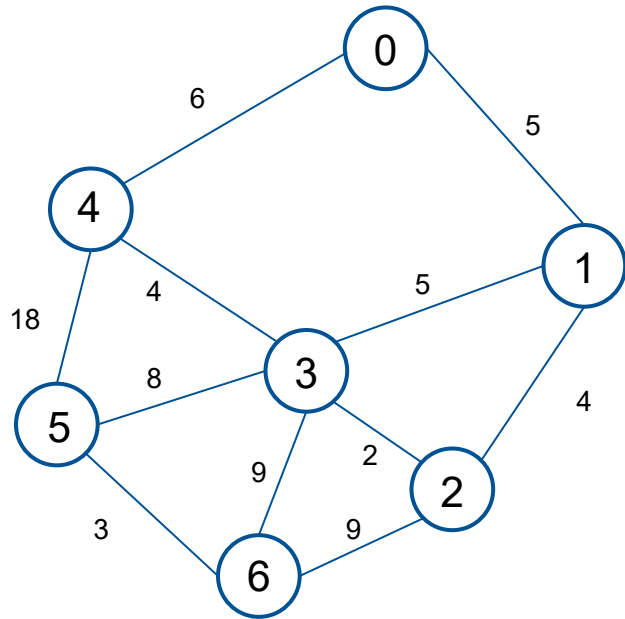
$$E = \{ (0,1), (1,0), (0,4), (4,0), \\ (1,2), (2,1), (1,3), (3,1), \\ (2,3), (3,2), (2,6), (6,2), \\ (3,4), (4,3), (3,5), (5,3), (3,6), (6,3), \\ (4,5), (5,4), \\ (5,6), (6,5) \}$$

c_{ij} = length of edge (i,j)

$x_{ij} \in \{0,1\}$ decision variable whether to use edge (i,j)

(Symmetric) Traveling Salesmen Problem (TSP)

$x_{ij} \in \{0,1\}$ decision variable whether to use edge (i,j)

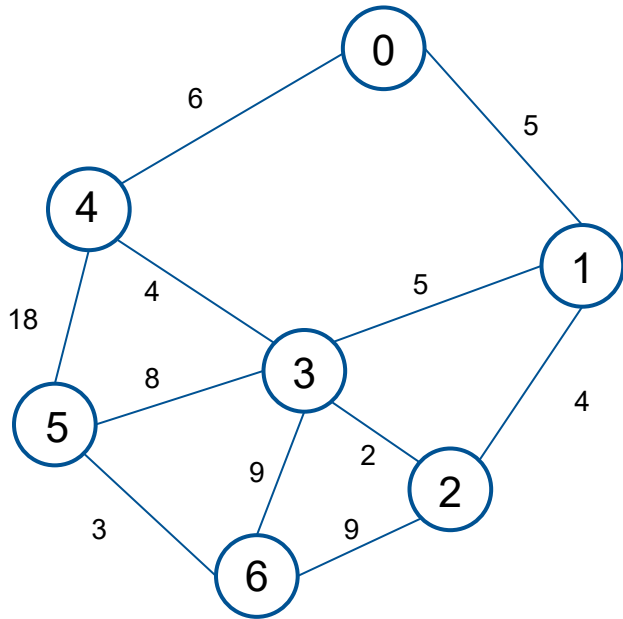


$$\min \sum_{(i,j) \in E} c_{ij} x_{ij}$$

s. t.

$$\sum_{i \in V: (i,j) \in E} x_{ij} + \sum_{i \in V: (j,i) \in E} x_{ji} = 2, \quad j \in V$$

(Symmetric) Traveling Salesmen Problem (TSP)



$$\min \sum_{(i,j) \in E} c_{ij} x_{ij} = 5x_{01} + 5x_{10} + 6x_{04} + 6x_{40} + 4x_{12} + 4x_{21} + 5x_{13} + 5x_{31} + 2x_{23} + 2x_{32} + 9x_{26} + 9x_{62} + 4x_{34} + 4x_{43} + 8x_{35} + 8x_{53} + 9x_{36} + 9x_{63} + 18x_{45} + 18x_{54} + 3x_{56} + 3x_{65}$$

$$\sum_{i \in V: (i,j) \in E} x_{ij} + \sum_{i \in V: (j,i) \in E} x_{ji} = 2, \quad j \in V$$

$$x_{10} + x_{40} + x_{01} + x_{04} = 2$$

$$x_{01} + x_{21} + x_{31} + x_{10} + x_{12} + x_{13} = 2$$

$$x_{12} + x_{32} + x_{62} + x_{21} + x_{23} + x_{26} = 2$$

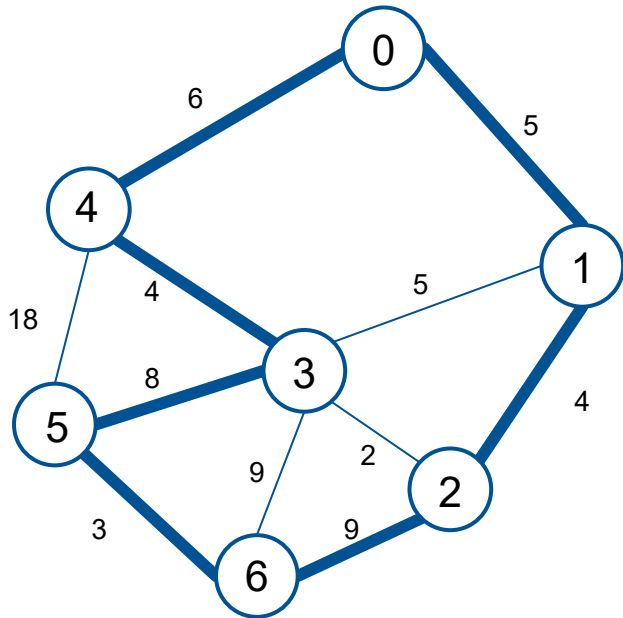
$$x_{13} + x_{23} + x_{43} + x_{53} + x_{63} + x_{31} + x_{32} + x_{34} + x_{35} + x_{36} = 2$$

$$x_{04} + x_{34} + x_{54} + x_{40} + x_{43} + x_{45} = 2$$

$$x_{35} + x_{45} + x_{65} + x_{53} + x_{54} + x_{56} = 2$$

$$x_{26} + x_{36} + x_{56} + x_{62} + x_{63} + x_{65} = 2$$

(Symmetric) Traveling Salesmen Problem (TSP)



$$\sum_{(i,j) \in E} c_{ij} x_{ij} = 5x_{01} + 5x_{10} + 6x_{04} + 6x_{40} + 4x_{12} + 4x_{21} + 5x_{13} + 5x_{31} + 2x_{23} + 2x_{32} + 9x_{26} + 9x_{62} + 4x_{34} + 4x_{43} + 8x_{35} + 8x_{53} + 9x_{36} + 9x_{63} + 18x_{45} + 18x_{54} + 3x_{56} + 3x_{65} = 39$$

$$\sum_{i \in V: (i,j) \in E} x_{ij} + \sum_{i \in V: (j,i) \in E} x_{ji} = 2, \quad j \in V$$

$$x_{10} + x_{40} + x_{01} + x_{04} = 2$$

$$x_{01} + x_{21} + x_{31} + x_{10} + x_{12} + x_{13} = 2$$

$$x_{12} + x_{32} + x_{62} + x_{21} + x_{23} + x_{26} = 2$$

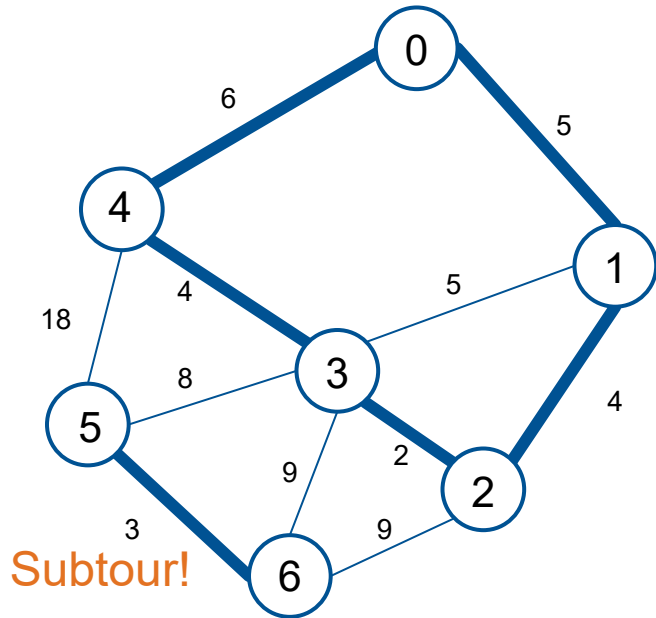
$$x_{13} + x_{23} + x_{43} + x_{53} + x_{63} + x_{31} + x_{32} + x_{34} + x_{35} + x_{36} = 2$$

$$x_{04} + x_{34} + x_{54} + x_{40} + x_{43} + x_{45} = 2$$

$$x_{35} + x_{45} + x_{65} + x_{53} + x_{54} + x_{56} = 2$$

$$x_{26} + x_{36} + x_{56} + x_{62} + x_{63} + x_{65} = 2$$

(Symmetric) Traveling Salesmen Problem (TSP)



$$\sum_{(i,j) \in E} c_{ij} x_{ij} = 5x_{01} + 5x_{10} + 6x_{04} + 6x_{40} + 4x_{12} + 4x_{21} + 5x_{13} + 5x_{31} + 2x_{23} + 2x_{32} + 9x_{26} + 9x_{62} + 4x_{34} + 4x_{43} + 8x_{35} + 8x_{53} + 9x_{36} + 9x_{63} + 18x_{45} + 18x_{54} + 3x_{56} + 3x_{65} = 27$$

$$\sum_{i \in V: (i,j) \in E} x_{ij} + \sum_{i \in V: (j,i) \in E} x_{ji} = 2, \quad j \in V$$

$$x_{10} + x_{40} + x_{01} + x_{04} = 2$$

$$x_{01} + x_{21} + x_{31} + x_{10} + x_{12} + x_{13} = 2$$

$$x_{12} + x_{32} + x_{62} + x_{21} + x_{23} + x_{26} = 2$$

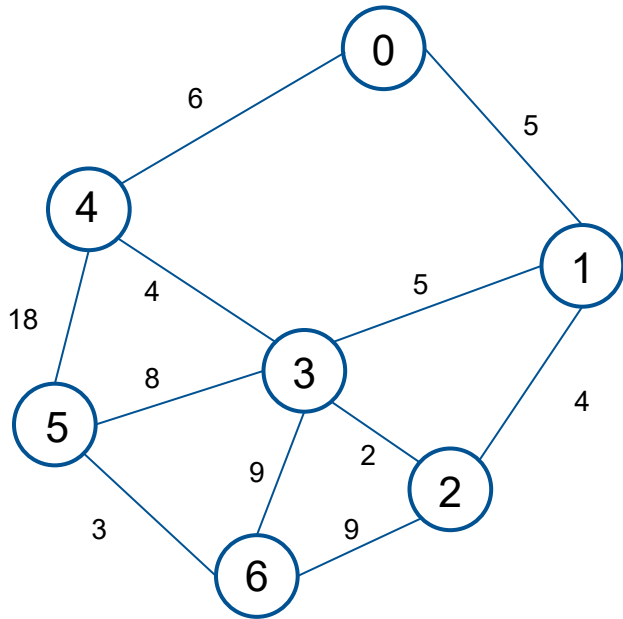
$$x_{13} + x_{23} + x_{43} + x_{53} + x_{63} + x_{31} + x_{32} + x_{34} + x_{35} + x_{36} = 2$$

$$x_{04} + x_{34} + x_{54} + x_{40} + x_{43} + x_{45} = 2$$

$$x_{35} + x_{45} + x_{65} + x_{53} + x_{54} + x_{56} = 2$$

$$x_{26} + x_{36} + x_{56} + x_{62} + x_{63} + x_{65} = 2$$

(Symmetric) Traveling Salesmen Problem (TSP)



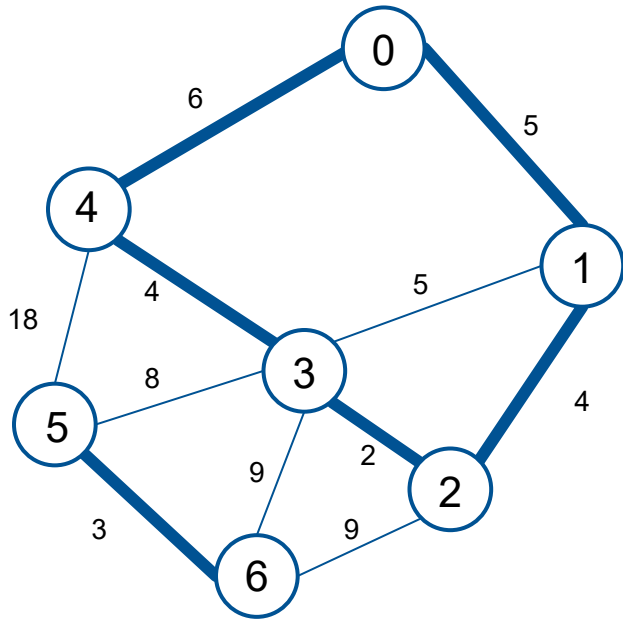
$$\min \sum_{(i,j) \in E} c_{ij} x_{ij}$$

s. t.

$$\sum_{i \in V: (i,j) \in E} x_{ij} + \sum_{i \in V: (j,i) \in E} x_{ji} = 2, \quad j \in V$$

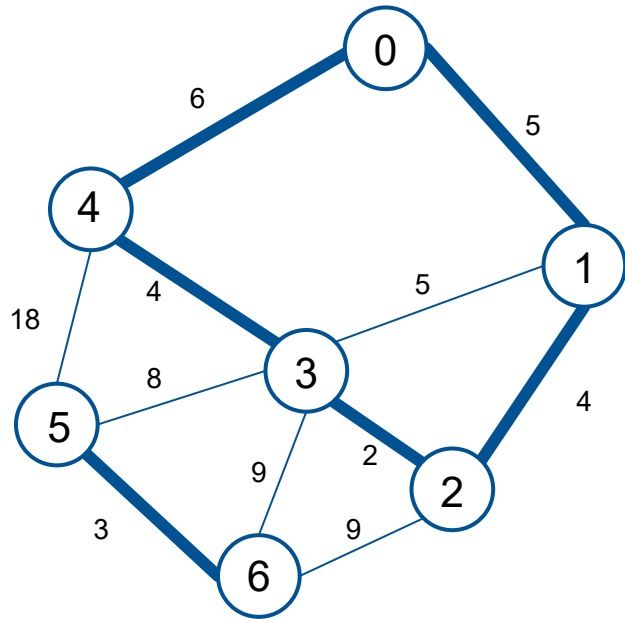
$$\sum_{(i,j) \in E: i \in S, j \notin S} x_{ij} + \sum_{(j,i) \in E: i \in S, j \notin S} x_{ji} \geq 2, \quad S \subset V, 2 \leq |S| \leq \left\lceil \frac{|V|}{2} \right\rceil$$

(Symmetric) Traveling Salesmen Problem (TSP)



$$\sum_{(i,j) \in E: i \in S, j \notin S} x_{ij} + \sum_{(j,i) \in E: i \in S, j \notin S} x_{ji} \geq 2, \quad S \subset V, 2 \leq |S| \leq \left\lceil \frac{|V|}{2} \right\rceil$$

(Symmetric) Traveling Salesmen Problem (TSP)



$$\sum_{(i,j) \in E: i \in S, j \notin S} x_{ij} + \sum_{(j,i) \in E: i \in S, j \notin S} x_{ji} \geq 2, \quad S \subset V, 2 \leq |S| \leq \left\lceil \frac{|V|}{2} \right\rceil$$

$$x_{53} + x_{54} + x_{62} + x_{63} + x_{26} + x_{36} + x_{35} + x_{45} = 0 < 2 \quad S = \{5, 6\}$$

(Symmetric) Traveling Salesmen Problem (TSP)

$$\min \sum_{(i,j) \in E} c_{ij} x_{ij}$$

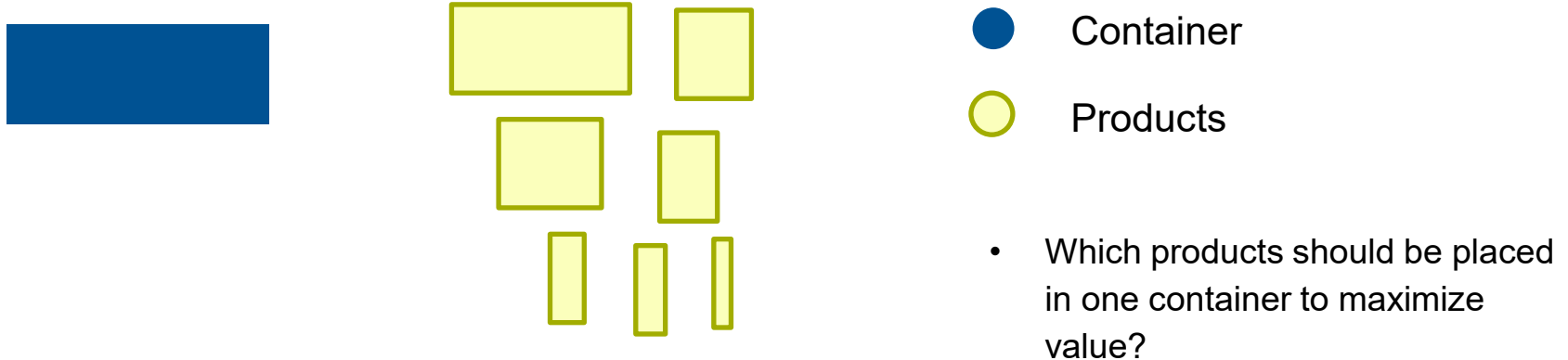
s. t.

$$\sum_{i \in V: (i,j) \in E} x_{ij} + \sum_{i \in V: (j,i) \in E} x_{ji} = 2, \quad j \in V$$

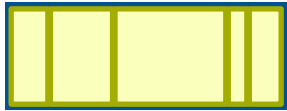
$$\sum_{(i,j) \in E: i \in S, j \notin S} x_{ij} + \sum_{(j,i) \in E: i \in S, j \notin S} x_{ji} \geq 2, \quad S \subset V, 2 \leq |S| \leq \left\lceil \frac{|V|}{2} \right\rceil$$

Knapsack Problem

Which products should be placed in one container?



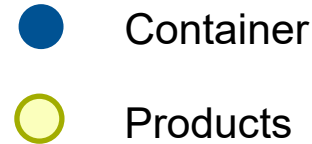
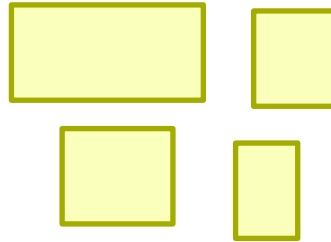
Which products should be placed in one container?



- Container
- Products

- Which products should be placed in one container to maximize value?

Which products should be placed in one container?



v_i value of product i

w_i width of product i

W width of container

} Formulate this problem as an optimization problem!

Which products should be placed in one container?

v_i value of product i

w_i width of product i

W width of container

$x_i \in \{0,1\}$

$$\max \sum_{i=1}^n v_i x_i$$

s. t.

$$\sum_{i=1}^n w_i x_i \leq W$$

Why is the knapsack a hard problem?

Combinatorial problem